Phase 4

4.1.1. Externally Sourced Code

```
public static String getMacaddress()
  {
    try {
      // Get the local host address
      InetAddress localHost = InetAddress.getLocalHost();
                // Get the network interface for the local host address
      NetworkInterface networkInterface =
NetworkInterface.getByInetAddress(localHost);
                // Get the MAC address
      byte[] macAddressBytes = networkInterface.getHardwareAddress();
      // Convert the MAC address bytes to a readable format
      StringBuilder macAddress = new StringBuilder();
      for (int i = 0; i < macAddressBytes.length; i++) {
        macAddress.append(String.format("%02X%s", macAddressBytes[i], (i <
macAddressBytes.length - 1) ? "-" : ""));
      }
      // Print the MAC address
      return "MAC Address: " + macAddress.toString();
    }
    catch (UnknownHostException | SocketException e)
    {
      return e.toString();
    }
  }//end of getMacaddress
```

```
//Class for Database management
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.time.LocalDate;


public class DBM {
   //Attributes
   //private static Connection conn;
   private static Connection dbconn;
   //private static Connection conn;
   private static PreparedStatement pst;
   static ResultSet rs;

   //Constructor
   public DBM(Connection Scon)
   {
      dbconn=Scon;

   }//end constructor

   public void DMBC(String URL)//checking if connection is valid
   {
      try
      {
         dbconn = DriverManager.getConnection(URL);
         System.out.println("connection good");
      }
      catch(SQLException ex)
      {
         System.err.println("File not found");
      }
   }//end
}//end of class
```

4.2. Explanation of Critical Algorithms

```
//Method to check login and log user into a window based on the role of the user
 public String CheckUserNamePassword(Connection con,String username,String
password,String role)
  {
     boolean bfound=false;
     String emname="";
     String comment="";
     try
     {
        Statement SQLM = con.createStatement();
        ResultSet rs = SQLM.executeQuery("Select  * From Employee Where
Username='"+username+"' AND Password='"+password+"'"+"AND Role='"+role+"'");
        while (rs.next())
        {
           emname  = rs.getString("EmployeeName");
           empid = rs.getInt("EmployeeID");
           String usrn = rs.getString("Username");//change to your database
           String pass = rs.getString("Password");
           String rolee  = rs.getString("Role");
           if (username.equals(usrn) && password.equals(pass) && role.equals(rolee))
              bfound=true;
        }//end loop
        if (bfound)
        {
           switch(role)
           {
              case "Admin":
                 AdminPane.setVisible(true);
                 LoginPane.setVisible(false);
                 getMacaddress();
                 Log(con, role, username, password,bfound);
                 comment=  "Welcome, "+emname;
                 break;
              case "Employee":
                 EmployeePane.setVisible(true);
                 LoginPane.setVisible(false);
                 getMacaddress();
                 Log(con, role, username, password,bfound);
```

```
comment= "Welcome, "+emname;
                break;

            }//end of switch
        }
        else
        {
            icount--;
            comment="login unsuccessful\n"+icount+" Attempts left";
            Log(con, role, username, password,bfound);
            if(icount==0)
            {
                comment="You have used up all your 3 attempts. Please Close the program, contact
admin if you cannot remeber your login info and then try again";
                try{
                    LoginPane.setClosed(true);
                }//end of try
                catch(PropertyVetoException ex)
                {
                    System.err.println(ex);
                }//end of catch
            }
        }//end of else
    }//end try
```

The algorithm checks the username, password, and role against stored data in the database. This ensures that only users with valid credentials can gain access to the system. Proper verification is crucial to prevent unauthorized access.

```
public static String getMacaddress()
  {
    try{
      // Get the local host address
      InetAddress localHost = InetAddress.getLocalHost();
                // Get the network interface for the local host address
      NetworkInterface networkInterface =
NetworkInterface.getByInetAddress(localHost);
                // Get the MAC address
      byte[] macAddressBytes = networkInterface.getHardwareAddress();
      // Convert the MAC address bytes to a readable format
      StringBuilder macAddress = new StringBuilder();
      for (int i = 0; i < macAddressBytes.length; i++) {
        macAddress.append(String.format("%02X%s", macAddressBytes[i], (i <
macAddressBytes.length - 1) ? "-" : ""));
      }
      // Print the MAC address
      return "MAC Address: " + macAddress.toString();
    }
    catch (UnknownHostException | SocketException e)
    {
      return e.toString();
    }
  }//end of getMacaddress
```

The MAC address is a unique identifier assigned to the network interface of each device. By retrieving and logging the MAC address, the program can uniquely identify the device from which the user is logging in. This helps ensure that the login is coming from a recognized device, adding an extra layer of security.

4.1.3. Advanced Techniques

```
LocalDate from,to;

    SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");

    String sfrom = sdf.format(dtechsr1from.getDate());

    from = LocalDate.parse(sfrom, DateTimeFormatter.ofPattern("dd/MM/yyyy"));




    String sto = sdf.format(dtechsr1to.getDate());

    to = LocalDate.parse(sto, DateTimeFormatter.ofPattern("dd/MM/yyyy"));


    String sql = "SELECT EmployeeName, DatePurchased, CustomerName, ProductName,
Total FROM Customers, Employee, Orders, Products WHERE
Products.ProductID=Orders.ProductID AND Employee.EmployeeID=Orders.OrderID AND
Customers.CustomerID=Orders.CustomerID";

    DBM dbm = new DBM(dbconn);

    Monitor(dbconn,tblmonitor,sql,from,to);
```

This Java code snippet is used for handling date inputs, formatting them, and running a database query to fetch records between those dates. Here's a detailed breakdown of the code:

1. Declaring Variables

LocalDate from, to;

SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");

LocalDate from, to: These are variables of type LocalDate that will store the start and end dates.

SimpleDateFormat sdf: An instance of SimpleDateFormat is created with the pattern dd/MM/yyyy to format dates in the day/month/year format.

2. Formatting and Parsing Start Date

String sfrom = sdf.format(dtechsr1from.getDate());

from = LocalDate.parse(sfrom, DateTimeFormatter.ofPattern("dd/MM/yyyy"));

dtechsr1from.getDate(): This method presumably retrieves the date from a date picker or some other UI component.
sfrom: The date is formatted as a string in the dd/MM/yyyy format.
from: The formatted string is then parsed into a LocalDate object using LocalDate.parse() and DateTimeFormatter.ofPattern("dd/MM/yyyy").

3. Formatting and Parsing End Date

String sto = sdf.format(dtechsr1to.getDate());
to = LocalDate.parse(sto, DateTimeFormatter.ofPattern("dd/MM/yyyy"));
dtechsr1to.getDate(): Similar to the start date, this retrieves the end date from the UI component.
sto: The end date is formatted as a string.
to: The formatted string is then parsed into a LocalDate object.

4. SQL Query Construction

String sql = "SELECT EmployeeName, DatePurchased, CustomerName, ProductName, Total FROM Customers, Employee, Orders, Products WHERE Products.ProductID=Orders.ProductID AND Employee.EmployeeID=Orders.OrderID AND Customers.CustomerID=Orders.CustomerID";
sql: This string contains an SQL query to retrieve data from multiple tables (Customers, Employee, Orders, Products).
The query selects the EmployeeName, DatePurchased, CustomerName, ProductName, and Total.
The tables are joined based on matching IDs (ProductID, OrderID, CustomerID).

5. Database and Monitoring Operations

DBM dbm = new DBM(dbconn);
Monitor(dbconn, tblmonitor, sql, from, to);
DBM dbm = new DBM(dbconn);: An instance of a database manager (DBM) is created, initialized with a database connection (dbconn). The DBM class likely encapsulates methods for executing queries and handling database interactions.
Monitor(dbconn, tblmonitor, sql, from, to);: This method is called to perform a monitoring operation, passing the database connection (dbconn), a table (tblmonitor), the SQL query (sql), and the date range (from, to).

```
public static String getMacaddress()
  {
     try {
        // Get the local host address
        InetAddress localHost = InetAddress.getLocalHost();
                    // Get the network interface for the local host address
        NetworkInterface networkInterface =
NetworkInterface.getByInetAddress(localHost);
                    // Get the MAC address
        byte[] macAddressBytes = networkInterface.getHardwareAddress();
        // Convert the MAC address bytes to a readable format
        StringBuilder macAddress = new StringBuilder();
        for (int i = 0; i < macAddressBytes.length; i++) {
           macAddress.append(String.format("%02X%s", macAddressBytes[i], (i <
macAddressBytes.length - 1) ? "-" : ""));
        }
        // Print the MAC address
        return "MAC Address: " + macAddress.toString();
     }
     catch (UnknownHostException | SocketException e)
     {
        return e.toString();
     }
  }//end of getMacaddress
```

This code is a Java method designed to retrieve the MAC (Media Access Control) address of the local machine and return it as a String. Here's a detailed breakdown:

Method Signature

public static String getMacaddress()

- **public**: This means the method is accessible from other classes.
- **static**: Indicates that the method belongs to the class rather than an instance of the class. This allows it to be called without creating an object of the class.
- **String**: The method returns a String.
- **getMacaddress**: The name of the method.

Try-Catch Block

```
try {
   // Code to execute
```

```
} catch (UnknownHostException | SocketException e) {
    return e.toString();
}
```

- The method attempts to execute the code within the try block. If an exception occurs, it will be caught, and the method will return the exception's string representation.

Main Functionality

1. **Getting the Local Host Address**

```
InetAddress localHost = InetAddress.getLocalHost();
```

   - InetAddress.getLocalHost() retrieves the IP address of the local machine.

2. **Getting the Network Interface**

```
NetworkInterface networkInterface = NetworkInterface.getByInetAddress(localHost);
```

   - NetworkInterface.getByInetAddress(localHost) gets the network interface associated with the local host's IP address.

3. **Getting the MAC Address**

```
byte[] macAddressBytes = networkInterface.getHardwareAddress();
```

   - networkInterface.getHardwareAddress() returns the MAC address as a byte array.

4. **Formatting the MAC Address**

```
StringBuilder macAddress = new StringBuilder();
for (int i = 0; i < macAddressBytes.length; i++) {
    macAddress.append(String.format("%02X%s", macAddressBytes[i], (i <
macAddressBytes.length - 1) ? "-" : ""));
}
```

   - A StringBuilder is used to create a human-readable MAC address.
   - The for loop iterates over each byte in the macAddressBytes array.
   - String.format("%02X%s", ...) converts each byte into a two-digit hexadecimal string.
   - If the current byte is not the last one, a hyphen (-) is added after it.

5. **Returning the MAC Address**

```
return "MAC Address: " + macAddress.toString();
```

   - The final MAC address is converted to a string and returned in the format "MAC Address: XX-XX-XX-XX-XX-XX".

Exception Handling

If there is an issue (e.g., the local host can't be found or the network interface can't be accessed), the method catches the UnknownHostException or SocketException and returns the exception's message as a string.

End of Method

The comment //end of getMacaddress marks the end of the method.

4.2.1 Evaluation of the Programmed Solution

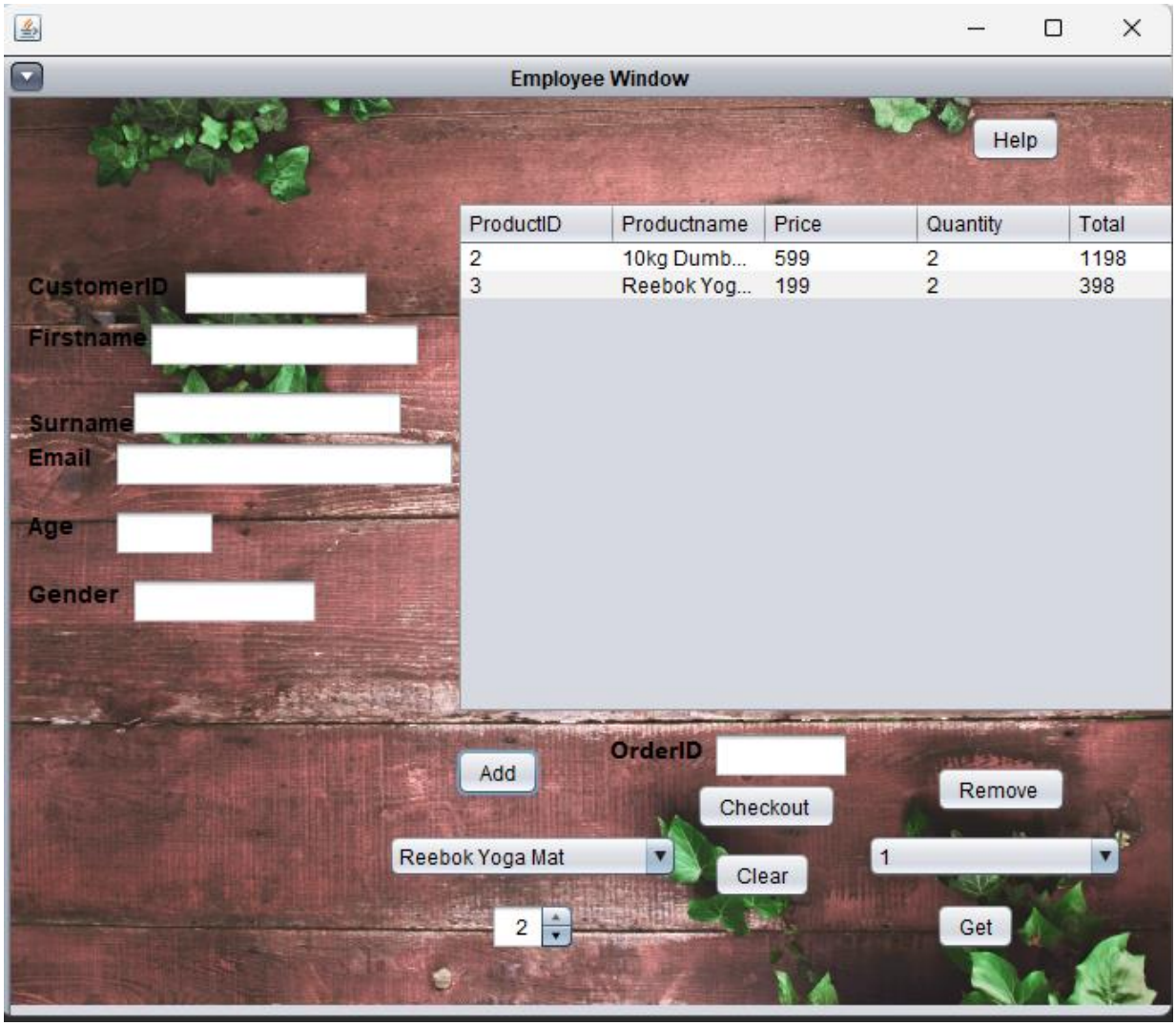
	Meet Requirements	Improvement/Shortfalls
A Generate button in the Admin window will allow the user(employee) to generate a report of daily, weekly, monthly, yearly transactions and to calculate the revenue made.	No	Instead of generating a report of daily, weekly, monthly, or yearly transactions, the user can generate a report based on selected dates
Create new button in the Admin window to allow the admin to create a new user based on the input received.	Yes	
Delete button in the Admin window to allow the admin to delete a user or a record from the database	Yes	
An Auto generate button in the Admin window to allow the admin to generate a username or password.	Yes	
An Update button in the Admin window to allow the admin to update a user's information.	Yes	
An Add button in the Employee window to allow the employee in the enterprise to add items in a cart(table) during transactions with customers.		
A Remove button in the Employee window to allow	Yes	

the employee to remove an item from the cart(table).		
A Checkout button in the Employee window to allow the employee to complete the transaction.	No	There is an issue with converting the date to a format that is acceptable by the database.

4.2.2. Functional Testing

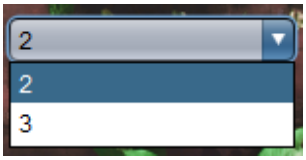
David Dwumah(tested software on August 6,2024)

I had zero issues logging into the employee window after I entered details(mei_chen, Employee, GoldenStar321). I selected dumb bell, the product I want to add to the table[cart] and for quantity I chose 2.

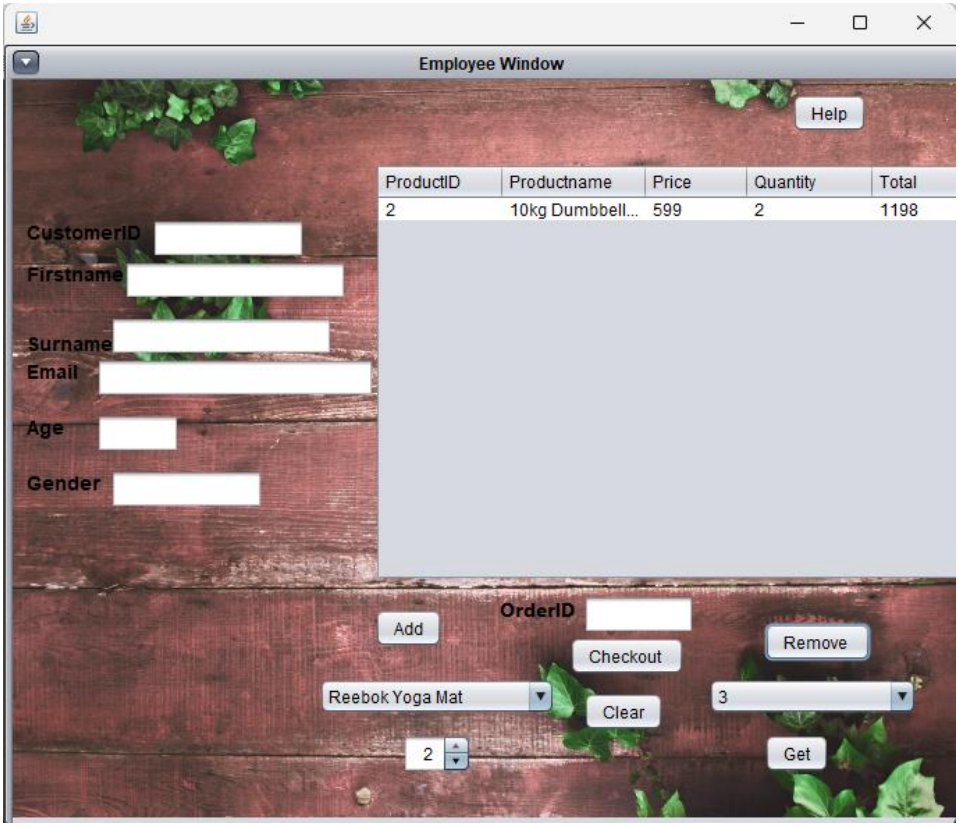


Next I wanted to remove the product with ID 3

So I selected the item from the combo box

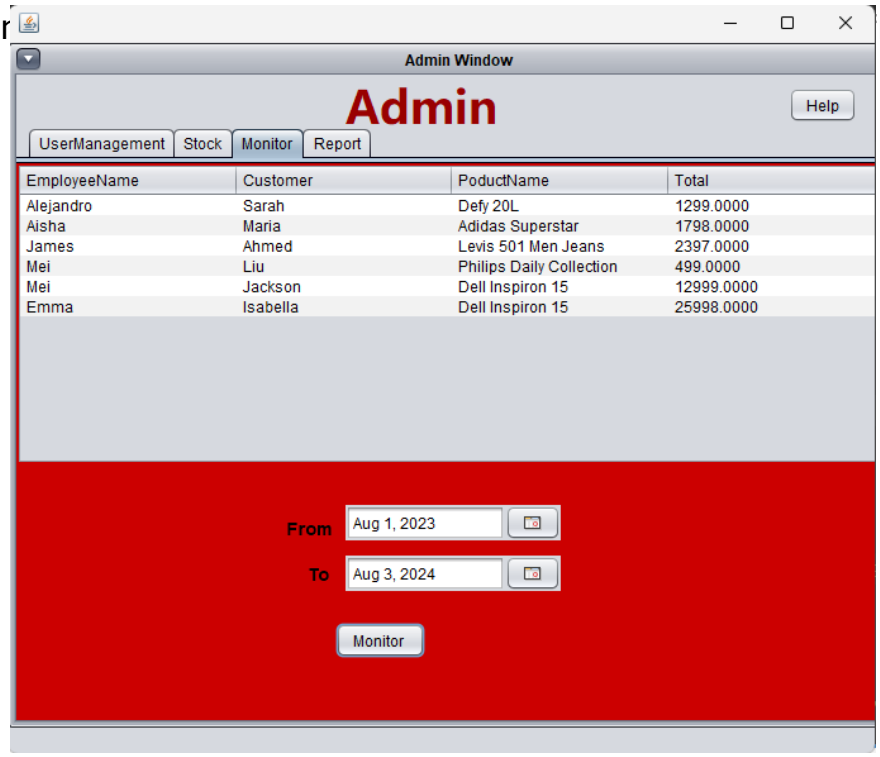


and clicked remove



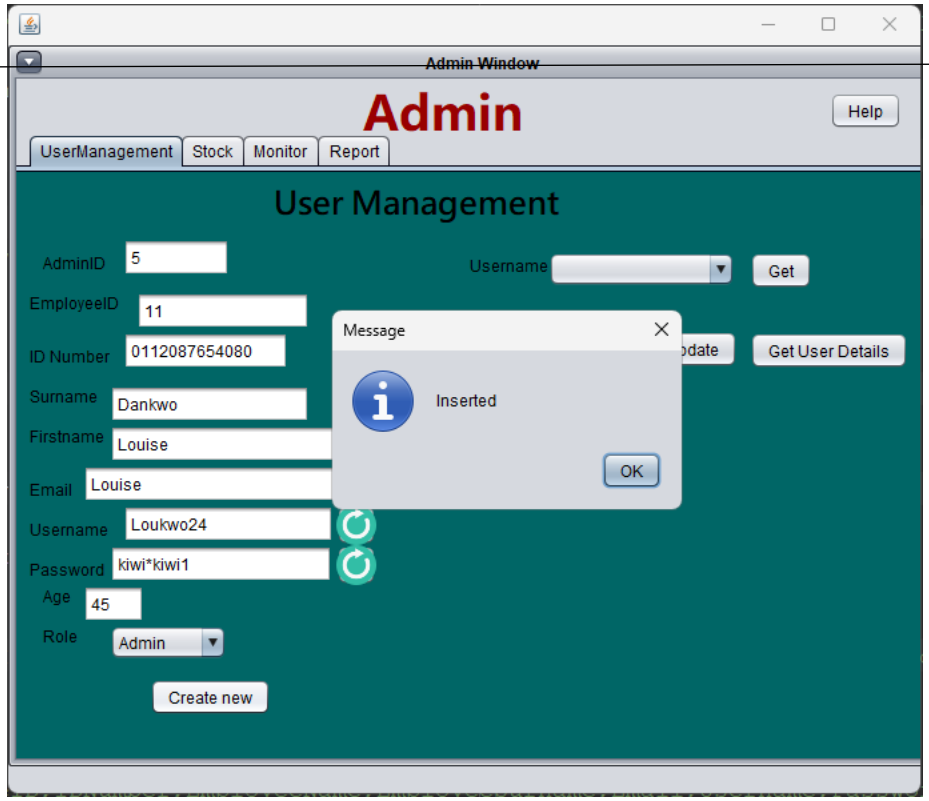
Amanfo Dwumah(tested software on August 6,2024)

I experience no issues logging into the Admin window after I entered details(emma_johnson, Admin, SecurePass123). Next, I clicked on the r



rom], Aug 1, 2024[to])

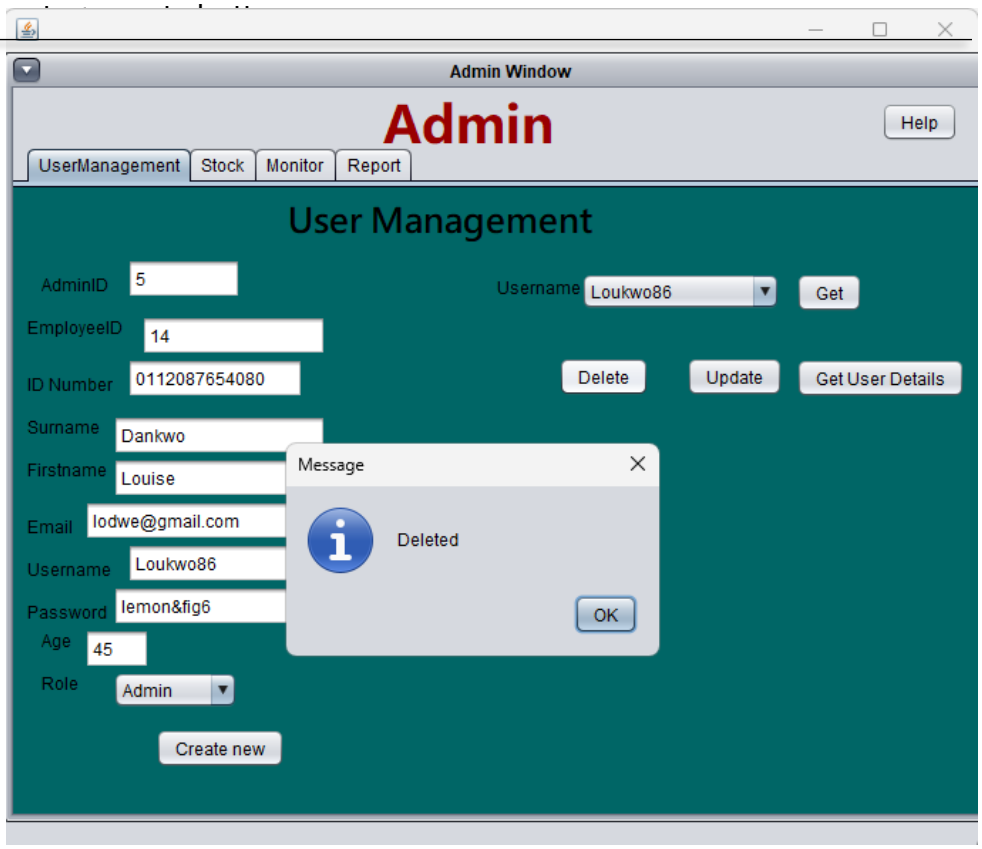
Next I went to the UserManagement tab and entered detail(5, 11, 011208765480, Dankwo, Louise, Louise, Loukwo24, kiwi*kiwi1, 45, Admin) Details[Loukwo24],



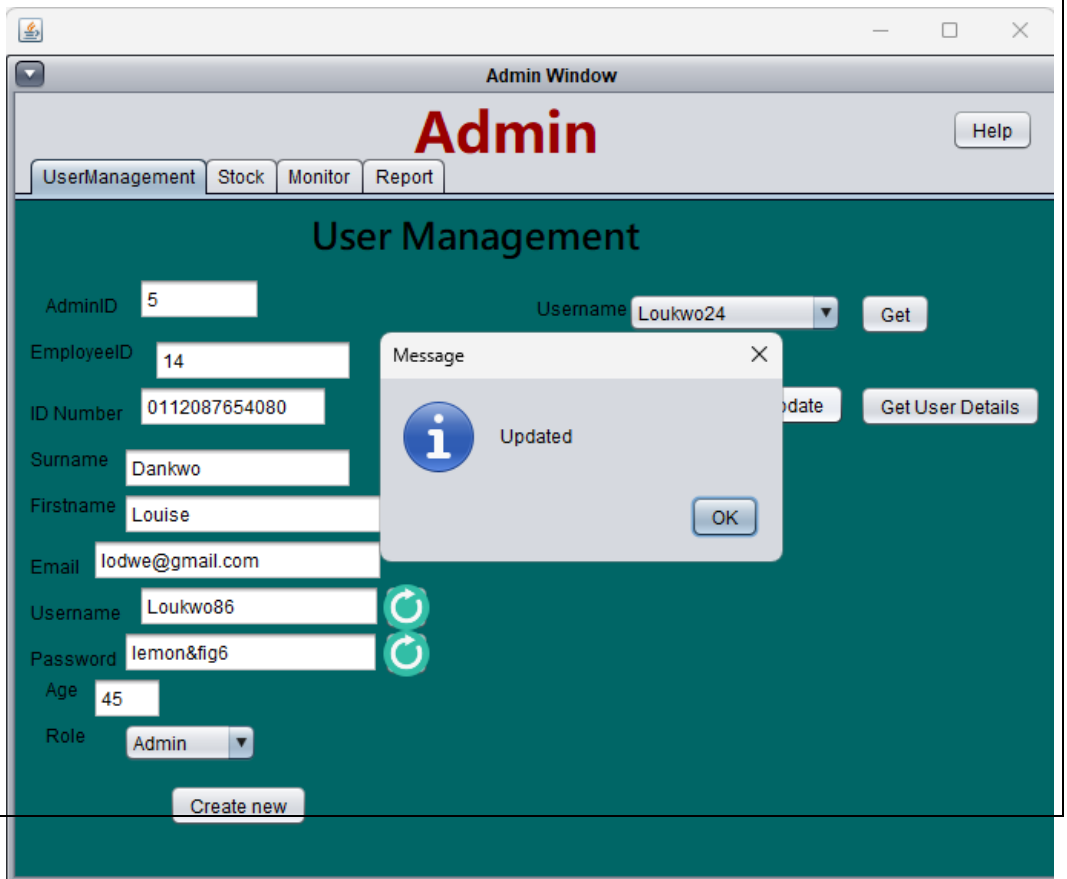
[kiwi*kiwi1] were

both auto generated by the

When I made a mistake with the email, I had to update the email address and auto generated a new username and password just for fun.



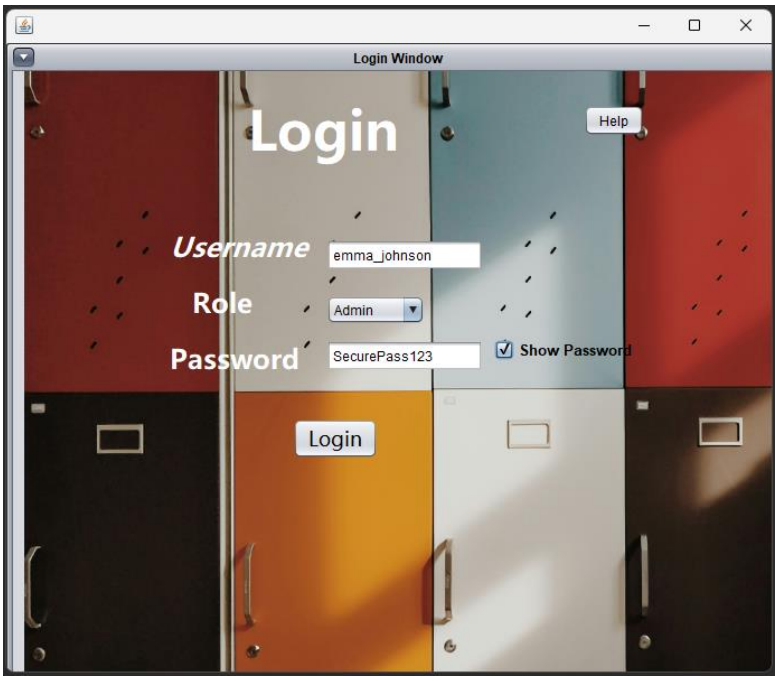
Now Louise was leaving the company, so I had to delete him from the record.



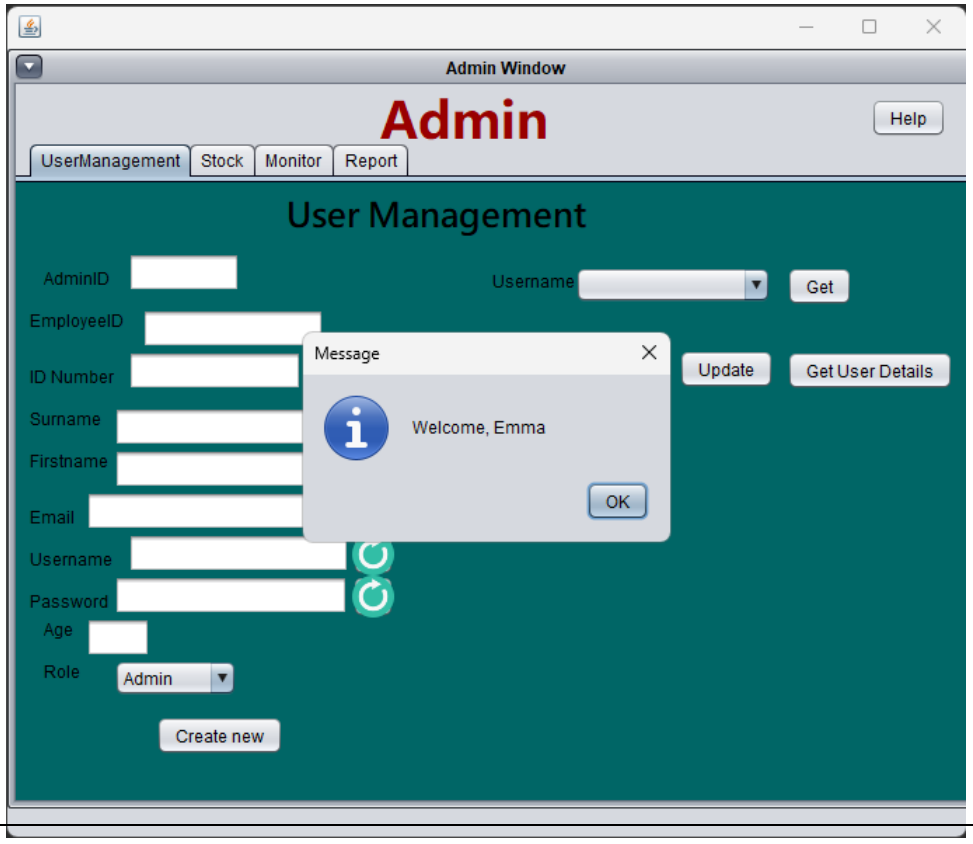
4.2.3 Test Plan and Results for TWO input variables

Standard data

Before Clicking Login

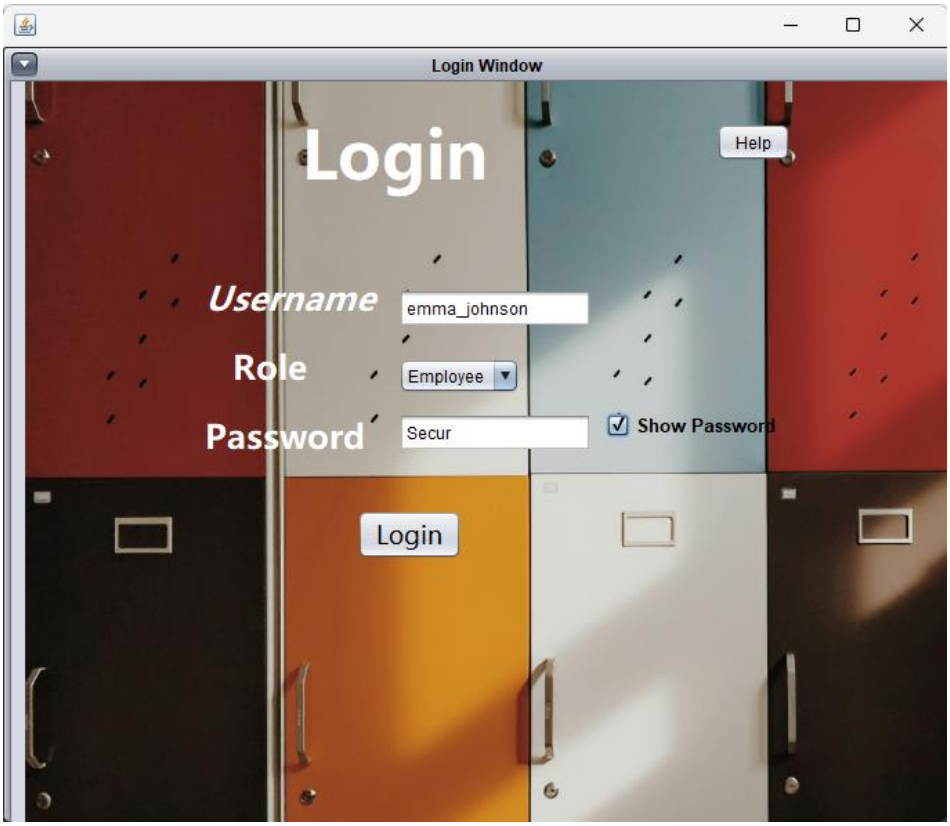


After Clicking Login

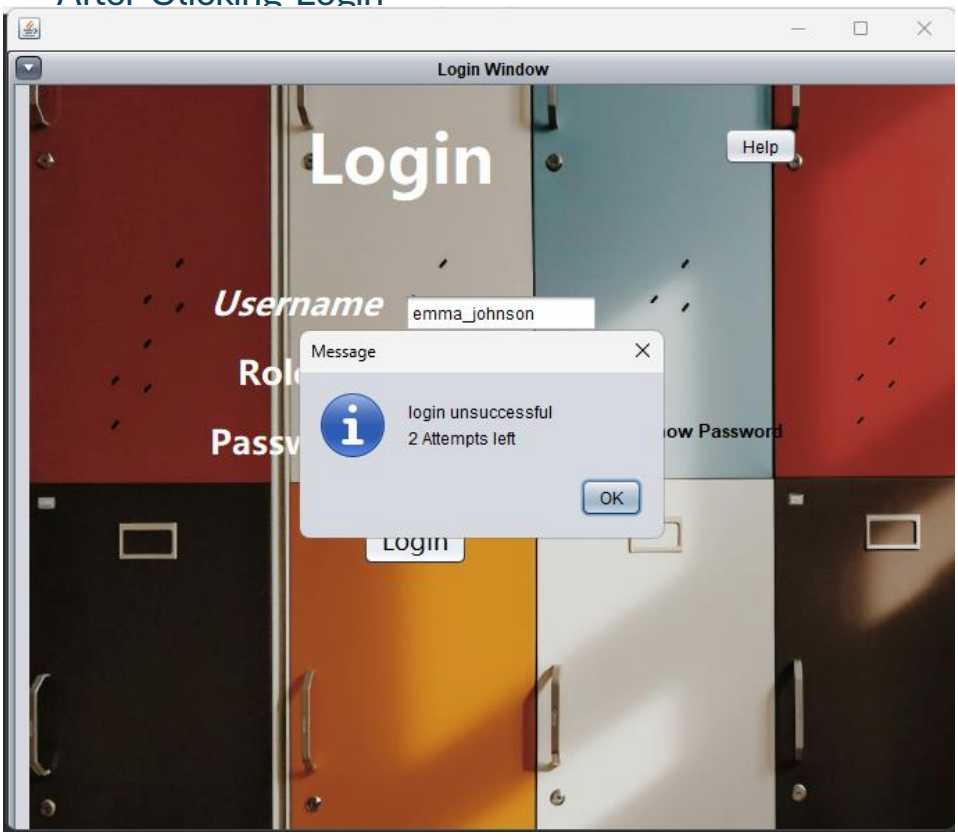


Abnormal Data

Before Clicking Login

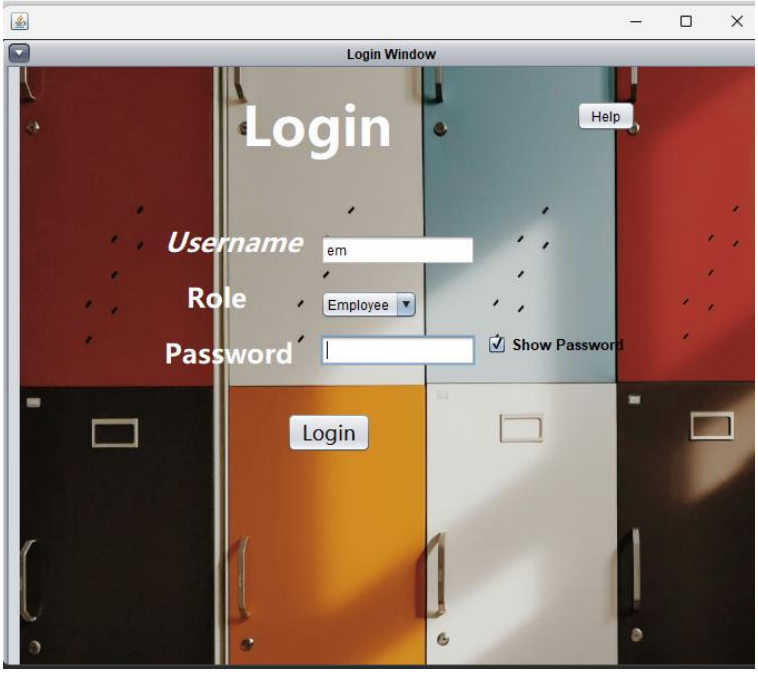
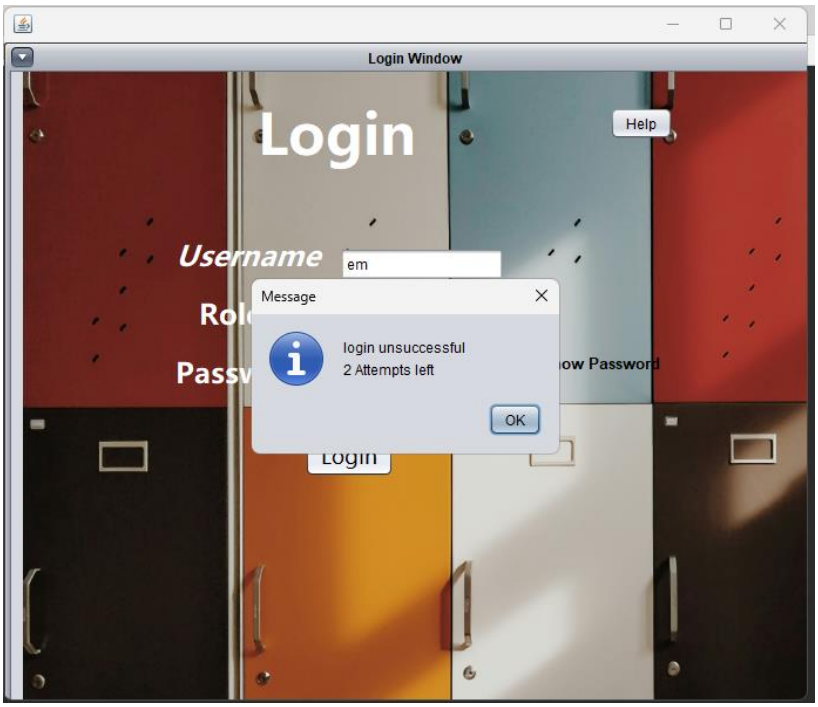


After Clicking Login



Extreme Date

Before Clicking Login



After Clicking Login